Day 10 — Joins

Combining Tables: Theory, Every Join Type & the Duplicate-Row Trap

---

1. Why Joins Exist (Theory)

In a relational database, data is deliberately split across multiple tables instead of being stored in one giant flat table. This is called **normalization** (you touched this in Day 21-22 conceptually, but it starts mattering practically right here).

For example, instead of repeating a customer's name and country on every single order row, a real system stores:

- A **customers** table - one row per customer
- An **orders** table - one row per order, referencing the customer via a `customer_id`

This avoids repeating the same customer name thousands of times, keeps data consistent (update the customer's country once, not on every order), and keeps tables smaller. A **JOIN** is how you bring that split data back together for a query.

Sample tables used throughout this document:

```
customers
---------
customer_id | customer_name | country
101         | Akash Sharma  | India
102         | Priya Singh   | India
103         | John Smith    | UK
104         | Maria Garcia  | Spain

orders
------
order_id | customer_id | order_date | amount
1        | 101         | 2026-01-05 | 1200
2        | 101         | 2026-01-18 | 800
3        | 102         | 2026-01-20 | 450
4        | 105         | 2026-01-22 | 300   -- customer_id 105 does not exist in customers
```

Notice: customer 103 and 104 have never placed an order, and order 4 belongs to a customer_id (105) that doesn't exist in the customers table. This is intentional - real data is rarely perfectly matched, and every join type below handles this mismatch differently.

---

2. INNER JOIN

Theory: An INNER JOIN returns only the rows where a match exists in *both* tables. If a row on either side has no matching counterpart, it is excluded entirely from the result. This is the default, most restrictive join.

```
SELECT o.order_id, c.customer_name, o.amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Result:

```
order_id | customer_name | amount
1        | Akash Sharma  | 1200
2        | Akash Sharma  | 800
3        | Priya Singh   | 450

-- Order 4 (customer_id 105) is EXCLUDED - no match in customers
-- Customers 103 and 104 are EXCLUDED - they have no orders
```

Key takeaway: Use INNER JOIN when you only care about records that exist on both sides - e.g., "show me orders and who placed them," where an order with no valid customer isn't useful to see.

---

3. LEFT JOIN (LEFT OUTER JOIN)

Theory: A LEFT JOIN returns *every* row from the left table, whether or not a match exists in the right table. When there's no match, the right table's columns are filled with NULL.

```
SELECT c.customer_name, o.order_id, o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Result:

```
customer_name | order_id | amount
Akash Sharma  | 1        | 1200
Akash Sharma  | 2        | 800
Priya Singh   | 3        | 450
John Smith    | NULL     | NULL   -- kept, even with no orders
Maria Garcia  | NULL     | NULL   -- kept, even with no orders
```

Key takeaway: LEFT JOIN is the most commonly used join in real analytics work - it answers questions like "show me all customers, including ones who never ordered anything," which INNER JOIN would silently hide.

---

4. RIGHT JOIN (RIGHT OUTER JOIN)

Theory: The mirror image of LEFT JOIN - returns every row from the right table, filling in NULL for the left table's columns when there's no match.

```
SELECT c.customer_name, o.order_id, o.amount
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

Result:

```
customer_name | order_id | amount
Akash Sharma  | 1        | 1200
Akash Sharma  | 2        | 800
Priya Singh   | 3        | 450
NULL          | 4        | 300    -- kept, even with no matching customer
```

Key takeaway: RIGHT JOIN is rarely used in practice - almost anything written with RIGHT JOIN can be rewritten as a LEFT JOIN by swapping which table comes first. Most style guides prefer LEFT JOIN everywhere for consistency and readability.

---

5. FULL OUTER JOIN

Theory: Combines LEFT and RIGHT JOIN - returns every row from both tables, matched where possible, with NULLs filled in on whichever side has no match.

```
SELECT c.customer_name, o.order_id, o.amount
FROM customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

Result:

```
customer_name | order_id | amount
Akash Sharma  | 1        | 1200
Akash Sharma  | 2        | 800
Priya Singh   | 3        | 450
John Smith    | NULL     | NULL   -- customer with no order
Maria Garcia  | NULL     | NULL   -- customer with no order
NULL          | 4        | 300    -- order with no matching customer
```

Key takeaway: Useful for finding mismatches on both sides at once - e.g., auditing data quality: "show me every customer with no orders AND every order with no valid customer, in one query."

---

6. SELF JOIN

Theory: Not a different SQL keyword - a self join is simply a table joined to *itself*, using two different aliases to treat it as if it were two separate tables. Used when rows in a table need to be compared to other rows in the same table.

Example: finding customers from the same country as each other.

```
SELECT a.customer_name AS customer_1, b.customer_name AS customer_2, a.country
FROM customers a
JOIN customers b
    ON a.country = b.country
    AND a.customer_id < b.customer_id  -- avoids matching a row with itself, and avoids duplica
```

Result:

```
customer_1    | customer_2   | country
Akash Sharma  | Priya Singh  | India
```

Key takeaway: The condition `a.customer_id < b.customer_id` ***is the standard trick to prevent a row from matching itself and to avoid getting every pair twice (once as A-B, once as B-A).***

---

7. CROSS JOIN

Theory: Returns every possible combination of rows from both tables (the mathematical Cartesian product) - there is no `ON` condition because nothing is being matched. If table A has 4 rows and table B has 3 rows, the result has 4 x 3 = 12 rows.

```
SELECT c.customer_name, p.product_name
FROM customers c
CROSS JOIN products p;

-- With 4 customers and, say, 3 products, this returns 4 x 3 = 12 rows -
-- every customer paired with every product.
```

Key takeaway: Genuinely useful cases are rare - mostly for generating combinations (e.g., every store paired with every product, to build a template that gets filled in later). Accidentally writing a CROSS JOIN by forgetting an ON condition is a common and expensive mistake on large tables.

---

8. The Duplicate-Row Problem Caused by Joins

Theory: A join matches rows based on the `ON` condition. If the join key is *not unique* on one side, a single row can match *multiple* rows on the other side - silently multiplying your data.

Example of the bug: imagine a customer has two orders, and you join customers to orders without realizing it:

```
-- customers has 1 row for Akash Sharma
-- orders has 2 rows for Akash Sharma (order 1 and order 2)

SELECT c.customer_name, c.country
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

Result (the bug):

```
customer_name | country
Akash Sharma  | India     -- from order 1
Akash Sharma  | India     -- from order 2 (duplicate!)
```

Akash Sharma now appears **twice**, even though you only asked for customer info. If you then ran `COUNT(*)` expecting "number of customers," you'd get an inflated, wrong answer - because the join silently duplicated the customer row for every matching order.

How to catch and fix it:

- Before joining, check whether the join key is unique on each side: `SELECT customer_id, COUNT(*) FROM orders GROUP BY customer_id HAVING COUNT(*) > 1;`
- If you only wanted customer info (not one row per order), aggregate the orders side first, or use `DISTINCT` deliberately - not as a silent band-aid.
- If duplication is expected (e.g., you want one row per order), that's not a bug - just be sure any COUNT/SUM afterward is calculated on the right table, not accidentally on the duplicated side.

Key takeaway: This is one of the most common real-world SQL bugs - a join that is technically correct but produces duplicated rows because a "one-to-many" relationship wasn't accounted for. Always ask: is the key I'm joining on actually unique on this side? before trusting a join's row count.

Day 10 Summary — Joins

Join Type	Returns
INNER JOIN	Only rows that match on both sides
LEFT JOIN	All rows from the left table, matched where possible
RIGHT JOIN	All rows from the right table, matched where possible
FULL OUTER JOIN	All rows from both tables, matched where possible
SELF JOIN	A table joined to itself, to compare its own rows
CROSS JOIN	Every possible combination of rows (Cartesian product)

Overall takeaway: Joins are how normalized, split-up data gets reassembled for analysis - but every join is a potential source of silently duplicated or silently dropped rows. The habit to build today: after every join, sanity-check the row count against what you expected, rather than assuming the join did exactly what you intended.